

## UNIT – V

### BRANCH AND BOUND -- THE METHOD

The design technique known as **branch and bound** is very similar to backtracking (seen in unit 4) in that it searches a tree model of the solution space and is applicable to a wide variety of discrete combinatorial problems.

Each node in the combinatorial tree generated in the last Unit defines a *problem state*. All paths from the root to other nodes define the **state space** of the problem.

**Solution states** are those problem states 's' for which the path from the root to 's' defines a tuple in the solution space. The leaf nodes in the combinatorial tree are the solution states.

**Answer states** are those solution states 's' for which the path from the root to 's' defines a tuple that is a member of the set of solutions (i.e., it satisfies the implicit constraints) of the problem. The tree organization of the solution space is referred to as the **state space tree**.

A node which has been generated and all of whose children have not yet been generated is called a **live node**.

The **live node** whose children are currently being generated is called the *E-node* (node being expanded).

A **dead node** is a generated node, which is not to be expanded further or all of whose children have been generated.

**Bounding functions** are used to kill live nodes without generating all their children.

Depth first node generation with bounding function is called backtracking. State generation methods in which the *E-node* remains the *E-node* until it is dead lead to **branch-and-bound method**.

The term branch-and-bound refers to all state space search methods in which all children of the *E-node* are generated before any other live node can become the *E-node*.

In branch-and-bound terminology breadth first search (BFS)- like state space search will be called FIFO (First In First Output) search as the list of live nodes is a first-in-first-out list (or queue).

A **D-search** (depth search) state space search will be called LIFO (Last In First Out) search, as the list of live nodes is a list-in-first-out list (or stack).

Bounding functions are used to help avoid the generation of sub trees that do not contain an answer node.

The branch-and-bound algorithms search a tree model of the solution space to get the solution. However, this type of algorithms is oriented more toward optimization. An algorithm of this type specifies a real-valued cost function for each of the nodes that appear in the search tree.

Usually, the goal here is to find a configuration for which the cost function is minimized. The branch-and-bound algorithms are rarely simple. They tend to be quite complicated in many cases.

### Least Cost (LC) search:

In both LIFO and FIFO Branch and Bound the selection rule for the next *E-node* is rigid and blind. The selection rule for the next *E-node* does not give any preference to a node that has a very good chance of getting the search to an answer node quickly. The search for an answer node can be speeded by using an "intelligent" ranking function  $c(\cdot)$  for live nodes. The next *E-node* is selected on the basis of this ranking function. The node  $x$  is assigned a rank using:

$c(x) = f(h(x)) + g(x)$  where,  $c(x)$  is the cost of  $x$ .

$h(x)$  is the cost of reaching  $x$  from the root and  $f(\cdot)$  is any non-decreasing function.

$g(x)$  is an estimate of the additional effort needed to reach an answer node from  $x$ .

A search strategy that uses a cost function  $c(x) = f(h(x)) + g(x)$  to select the next *E-node* would always choose for its next *E-node* a live node with least LC-search (Least Cost search)  $c(\cdot)$  is called a

BFS and D-search are special cases of LC-search. If  $g(x) = 0$  and  $f(h(x)) = \text{level of node } x$ , then an LC search generates nodes by levels. This is eventually the same as a BFS. If  $f(h(x)) = 0$  and essentially a D-search.

$g(x) > g(y)$  whenever  $y$  is a child of  $x$ , then the search is

An LC-search coupled with bounding functions is called an LC-branch and bound search. We associate a cost  $c(x)$  with each node  $x$  in the state space tree. It is not possible to easily compute the function  $c(x)$ . So we compute an estimate  $c(x)$  of  $c(x)$ .

### Control Abstraction for LC-Search:

Let  $t$  be a state space tree and  $c()$  a cost function for the nodes in  $t$ . If  $x$  is a node in  $t$ , then  $c(x)$  is the minimum cost of any answer node in the subtree with root  $x$ . Thus,  $c(t)$  is the cost of a minimum-cost answer node in  $t$ .

A heuristic  $c(.)$  is used to estimate  $c()$ . This heuristic should be easy to compute and generally has the property that if  $x$  is either an answer node or a leaf node, then  $c(x) = c(x)$ .

LC-search uses  $c$  to find an answer node. The algorithm uses two functions  $\text{Least}()$  and  $\text{Add}()$  to delete and add a live node from or to the list of live nodes, respectively.  $\text{Least}()$  finds a live node with least  $c()$ . This node is deleted from the list of live nodes and returned.

$\text{Add}(x)$  adds the new live node  $x$  to the list of live nodes. The list of live nodes be implemented as a min-heap.

Algorithm  $\text{LCSearch}$  outputs the path from the answer node it finds to the root node  $t$ . This is easy to do if with each node  $x$  that becomes live, we associate a field  $\text{parent}$  which gives the parent of node  $x$ . When the answer node  $g$  is found, the path from  $g$  to  $t$  can be determined by following a sequence of parent values starting from the current E-node (which is the parent of  $g$ ) and ending at node  $t$ .

Listnode = record

```
{
    Listnode * next, *parent; float cost;
}
```

Algorithm  $\text{LCSearch}(t)$

```
{ //Search t for an answer node
    if *t is an answer node then output *t and return;
    E := t; //E-node.
    initialize the list of live nodes to be empty;
    repeat
    {
        for each child x of E do
        {
            if x is an answer node then output the path from x to t and return;
            Add (x); //x is a new live node.
            (x → parent) := E; // pointer for path to root
        }
        if there are no more live nodes then
        {
            write ("No answer node");
            return;
        }
        E := Least();
    } until (false);
}
```

}

The root node is the first, E-node. During the execution of LC search, this list contains all live nodes except the E-node. Initially this list should be empty. Examine all the children of the E-node, if one of the children is an answer node, then the algorithm outputs the path from  $x$  to  $t$  and terminates. If the child of  $E$  is not an answer node, then it becomes a live node. It is added to the list of live nodes and its parent field set to  $E$ . When all the children of  $E$  have been generated,  $E$  becomes a dead node. This happens only if none of  $E$ 's children is an answer node. Continue the search further until no live nodes found. Otherwise, Least(), by definition, correctly chooses the next E-node and the search continues from here. LC search terminates only when either an answer node is found or the entire state space tree has been generated and searched.

### Bounding:

A branch and bound method searches a state space tree using any search mechanism in which all the children of the E-node are generated before another node becomes the E-node. We assume that each answer node  $x$  has a cost  $c(x)$  associated with it and that a minimum-cost answer node is to be found. Three common search strategies are FIFO, LIFO, and LC. The three search methods differ only in the selection rule used to obtain the next E-node.

A good bounding helps to prune efficiently the tree, leading to a faster exploration of the solution space. A cost function  $c(.)$  such that  $c(x) < c(x)$  is used to provide lower bounds on solutions obtainable from any node  $x$ . If upper is an upper bound on the cost of a minimum-cost solution, then all live nodes  $x$  with  $c(x) > c(x) > \text{upper}$ . The starting value for upper can be obtained by some heuristic or can be set to  $\infty$ .

As long as the initial value for upper is not less than the cost of a minimum-cost answer node, the above rules to kill live nodes will not result in the killing of a live node that can reach a minimum-cost answer node. Each time a new answer node is found, the value of upper can be updated. Branch-and-bound algorithms are used for optimization problems where, we deal directly only with minimization problems. A maximization problem is easily converted to a minimization problem by changing the sign of the objective function. To formulate the search for an optimal solution for a least-cost answer node in a state space tree, it is necessary to define the cost function  $c(.)$ , such that  $c(x)$  is minimum for all nodes representing an optimal solution. The easiest way to do this is to use the objective function itself for  $c(.)$ .

- For nodes representing feasible solutions,  $c(x)$  is the value of the objective function for that feasible solution.
- For nodes representing infeasible solutions,  $c(x) = \infty$ .
- For nodes representing partial solutions,  $c(x)$  is the cost of the minimum-cost node in the subtree with root  $x$ .

Since,  $c(x)$  is generally hard to compute, the branch-and-bound algorithm will use an estimate  $c(x)$  such that  $c(x) < c(x)$  for all  $x$ .

### FIFO Branch and Bound:

A FIFO branch-and-bound algorithm for the job sequencing problem can begin with  $\text{upper} = \infty$  as an upper bound on the cost of a minimum-cost answer node. Starting with node 1 as the E-node and using the variable tuple size formulation of Figure 8.4, nodes 2, 3, 4, and 5 are generated. Then  $u(2) = 19$ ,  $u(3) = 14$ ,  $u(4) = 18$ , and  $u(5) = 21$ .

The variable upper is updated to 14 when node 3 is generated. Since  $c(4)$  and  $c(5)$  are greater than upper, nodes 4 and 5 get killed. Only nodes 2 and 3 remain alive.

Node 2 becomes the next E-node. Its children, nodes 6, 7 and 8 are generated. Then  $u(6) = 9$  and so upper is updated to 9. The cost gets killed. Node 8 is infeasible and so it is killed.  $c(7) = 10 > \text{upper}$  and node 7

Next, node 3 becomes the E-node. Nodes 9 and 10 are now generated. Then  $u(9) = 8$  and so upper becomes 8. The cost  $c(10) = 11 > \text{upper}$ , and this node is killed.

The next E-node is node 6. Both its children are infeasible. Node 9's only child is also infeasible. The minimum-cost answer node is node 9. It has a cost of 8.

When implementing a FIFO branch-and-bound algorithm, it is not economical to kill live nodes with  $c(x) > \text{upper}$  each time upper is updated. This is so because live nodes are in the queue in the order in which they were generated. Hence, nodes with  $c(x) > \text{upper}$  are distributed in some random way in the queue. Instead, live nodes with  $c(x) > \text{upper}$  can be killed when they are about to become E-nodes. The FIFO-based branch-and-bound algorithm with an appropriate  $c(\cdot)$  and  $u(\cdot)$  is called FIFOB.

### LC Branch and Bound:

An LC Branch-and-Bound search of the tree of Figure 8.4 will begin with  $\text{upper} = \infty$  and node 1 as the first E-node. When node 1 is expanded, nodes 2, 3, 4 and 5 are generated in that order. As in the case of FIFOB, upper is updated to 14 when node 3 is generated and nodes 4 and 5 are killed as  $c(4) > \text{upper}$  and  $c(5) > \text{upper}$ . Node 2 is the next E-node as  $c(2) = 0$  and  $c(3) = 5$ . Nodes 6, 7 and 8 are generated and upper is updated to 9 when node 6 is generated. So, node 7 is killed as  $c(7) = 10 > \text{upper}$ . Node 8 is infeasible and so killed. The only live nodes now are nodes 3 and 6. Node 6 is the next E-node as  $c(6) = 0 < c(3)$ . Both its children are infeasible. Node 3 becomes the next E-node. When node 9 is generated, upper is updated to 8 as  $u(9) = 8$ . So, node 10 with  $c(10) = 11$  is killed on generation. Node 9 becomes the next E-node. Its only child is infeasible. No live nodes remain. The search terminates with node 9 representing the minimum-cost answer node.

2      3

The path =  $1 \rightarrow 3 \rightarrow 9 = 5 + 3 = 8$

## TRAVELLING SALESMAN PROBLEM

### INTRODUCTION:

It is algorithmic procedures similar to backtracking in which a new branch is chosen and is there (bound there) until new branch is choosing for advancing.

This technique is implemented in the traveling salesman problem [TSP] which are asymmetric ( $C_{ij} \neq C_{ji}$ ) where this technique is an effective procedure.

### STEPS INVOLVED IN THIS PROCEDURE ARE AS FOLLOWS:

STEP 0: Generate cost matrix C [for the given graph g]

STEP 1: [ROW REDUCTION] For all rows do step 2

STEP 2: Find least cost in a row and negate it with rest of the elements.

$\text{Row}(M) = [M_{ij} - \min\{M_{ij} / 1 \leq j \leq n\}]$  where  $M_{ij} \neq \infty$

The minimum value of the row is considered and by using that value the entire row is reduced.

**STEP 3:[COLUMN REDUCTION]** Use cost matrix- Row reduced one for all columns do STEP 4.

**STEP 4:** Find least cost in a column and negate it with rest of the elements.

The cost of the matrix M is reduced by minimum value

$$\text{Col}(M) = [M_{ij} - \min\{M_{ji} / 1 \leq j \leq n\}] \quad \text{where } M_{ij} < \infty$$

**STEP 5:** Preserve cost matrix C [which row reduced first and then column reduced] for the  $i^{\text{th}}$  time.

**STEP 6:** Enlist all edges (i, j) having cost = 0.

**STEP 7:** Calculate effective cost of the edges.  $\sum (i, j) = \text{least cost in the } i^{\text{th}} \text{ row excluding } (i, j) + \text{least cost in the } j^{\text{th}} \text{ column excluding } (i, j).$

**STEP 8:** Compare all effective cost and pick up the largest I. If two or more have same cost then arbitrarily choose any one among them.

**STEP 9:** Delete (i, j) means delete  $i^{\text{th}}$  row and  $j^{\text{th}}$  column change (j, i) value to infinity. (Used to avoid infinite loop formation) If (i,j) not present, leave it.

**STEP 10:** Repeat step 1 to step 9 until the resultant cost matrix having order of  $2 \times 2$  and reduce it. (Both R.R and C.C)

**STEP 11:** Use preserved cost matrix  $C_n, C_{n-1} \dots C_1$ . Choose an edge [i, j] having value =0, at the first time for a preserved matrix and leave that matrix.

**STEP 12:** Use result obtained in Step 11 to generate a complete tour.

Example 1: consider the following the matrix

$$\begin{bmatrix} \infty & 20 & 30 & 10 & 11 \\ 15 & \infty & 16 & 4 & 2 \\ 3 & 5 & \infty & 2 & 4 \\ 19 & 6 & 18 & \infty & 3 \\ 16 & 4 & 7 & 16 & \infty \end{bmatrix}$$

**Solution:**

Minimum cost of each row is

$$\begin{bmatrix} \infty & 20 & 30 & 10 & 11 \\ 15 & \infty & 16 & 4 & 2 \\ 3 & 5 & \infty & 2 & 4 \\ 19 & 6 & 18 & \infty & 3 \\ 16 & 4 & 7 & 16 & \infty \end{bmatrix} \begin{matrix} 10 \\ 2 \\ 2 \\ 3 \\ 4 \end{matrix}$$

The total minimum cost is  $\rightarrow 10+2+2+3+4=21$

Row minimizations

$$\begin{bmatrix} \infty - 10 & 20 - 10 & 30 - 10 & 10 - 10 & 11 - 10 \\ 15 - 2 & \infty - 2 & 16 - 2 & 4 - 2 & 2 - 2 \\ 3 - 2 & 5 - 2 & \infty - 2 & 2 - 2 & 4 - 2 \\ 19 - 3 & 6 - 3 & 18 - 3 & \infty - 3 & 3 - 3 \\ 16 - 4 & 4 - 4 & 7 - 4 & 16 - 4 & \infty - 4 \end{bmatrix} \rightarrow \begin{bmatrix} \infty & 10 & 20 & 0 & 10 \\ 13 & \infty & 14 & 2 & 0 \\ 1 & 3 & \infty & 0 & 2 \\ 16 & 3 & 15 & \infty & 0 \\ 12 & 0 & 3 & 12 & \infty \end{bmatrix}$$

Minimum cost of each column by not considering 0 or  $\infty$  ( $i \rightarrow$  ignore)

$$\begin{array}{ccccc}
 \infty & 10 & 20 & 0 & 10 \\
 13 & \infty & 14 & 2 & 0 \\
 \mathbf{1} & 3 & \infty & 0 & 2 \\
 16 & 3 & 15 & \infty & 0 \\
 12 & 0 & \mathbf{3} & 12 & \infty \\
 & 1 & i & 3 & i & i
 \end{array}$$

The total cost is  $1+3=4$

Column minimization

$$\begin{array}{ccccc}
 \infty & 10 & 20-3 & 0 & 10 \\
 13-1 & \infty & 14-3 & 2 & 0 \\
 \mathbf{1}-1 & 3 & \infty & 0 & 2 \\
 16-1 & 3 & 15-3 & \infty & 0 \\
 12-1 & 0 & \mathbf{3}-3 & 12 & \infty
 \end{array}
 \rightarrow
 \begin{array}{ccccc}
 \infty & 10 & 17 & 0 & 10 \\
 12 & \infty & 11 & 2 & 0 \\
 0 & 3 & \infty & 0 & 2 \\
 15 & 3 & 12 & \infty & 0 \\
 11 & 0 & 0 & 12 & \infty
 \end{array}$$

Then the resultant matrix is

$$\begin{array}{ccccc}
 \infty & 20 & 30 & 10 & 11 \\
 15 & \infty & 16 & 4 & 2 \\
 3 & 5 & \infty & 2 & 4 \\
 19 & 6 & 18 & \infty & 3 \\
 16 & 4 & 7 & 16 & \infty
 \end{array}
 \rightarrow
 \begin{array}{ccccc}
 \infty & 10 & 17 & 0 & 10 \\
 12 & \infty & 11 & 2 & 0 \\
 0 & 3 & \infty & 0 & 2 \\
 15 & 3 & 12 & \infty & 0 \\
 11 & 0 & 0 & 12 & \infty
 \end{array}$$

Thus the total reduced cost will be  $\rightarrow \{\text{cost}(\text{Row}(M)) + \text{cost}(\text{Col}(M))\} \rightarrow 21+4 \rightarrow 25$

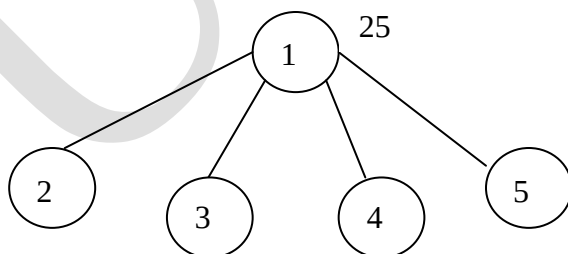
**Dynamic reduction:** We obtain the total reduced cost from the above statement as 25. By using the dynamic reduction we can make the choice of edge  $i \rightarrow j$  with the optimum cost.

Steps :

1. Draw the state space with optimum cost at root node
2. Obtain the cost of matrix for the path  $i \rightarrow j$  by making  $i^{\text{th}}$  row and  $j^{\text{th}}$  column entries as  $\infty$  and set  $M[i][j] = \infty$
3. Cost of the corresponding node 'x' with path l,j is optimum cost + reduced cost +  $M[i][j]$
4. Set node with minimum cost as E-node and generate its children .Repeat step 1 to 4 for the completing the tour with optimum cost.

Then the tree becomes  $\rightarrow$

The state space tree starts with optimum cost



Here 1 is the starting vertex with the optimum cost as 25, where as 2,3,4 and 5 are its children , we need to find the cost of paths [1,2], [1,3], [1,4], [1,5], by considering the minimum cost

**Path [1,2] :** make  $1^{\text{st}}$  row and  $2^{\text{nd}}$  column as  $\infty$  and set  $M[2,1] = \infty$  (no back tracking allowed )

$$\begin{bmatrix} \infty & \infty & \infty & \infty & \infty \\ \infty & \infty & 11 & 2 & 0 \\ 0 & \infty & \infty & 0 & 2 \\ 15 & \infty & 12 & \infty & 0 \\ 11 & \infty & 0 & 12 & \infty \end{bmatrix}$$

As each row and column as '0', the reduced cost will be '0'

→ Optimum cost + reduced cost + old value M[1,2]

→ 25+0+10

→ 35

**Path [1,3] :** make 1<sup>st</sup> row and 3<sup>rd</sup> column as  $\infty$  and set M[3,1]=  $\infty$  (no back tracking allowed )

$$\begin{bmatrix} \infty & \infty & \infty & \infty & \infty \\ 12 & \infty & \infty & 2 & 0 \\ \infty & 3 & \infty & 0 & 2 \\ 15 & 3 & \infty & \infty & 0 \\ \mathbf{11} & 0 & \infty & 12 & \infty \end{bmatrix}$$

As each row and column doesn't have '0', here the minimum cost will be 11 from the 1<sup>st</sup> column

$$\begin{bmatrix} \infty & \infty & \infty & \infty & \infty \\ 12 - 11 & \infty & \infty & 2 & 0 \\ \infty & 3 & \infty & 0 & 2 \\ 15 - 11 & 3 & \infty & \infty & 0 \\ 11 - 11 & 0 & \infty & 12 & \infty \end{bmatrix} \rightarrow \begin{bmatrix} \infty & \infty & \infty & \infty & \infty \\ 1 & \infty & \infty & 2 & 0 \\ \infty & 3 & \infty & 0 & 2 \\ 4 & 3 & \infty & \infty & 0 \\ 0 & 0 & \infty & 12 & \infty \end{bmatrix}$$

→ Optimum cost + reduced cost + old value M[1,3]

→ 25+11+17

→ 53

**Path [1,4] :** make 1<sup>st</sup> row and 4<sup>th</sup> column as  $\infty$  and set M[4,1]=  $\infty$  (no back tracking allowed )

$$\begin{bmatrix} \infty & \infty & \infty & \infty & \infty \\ 12 & \infty & 11 & \infty & 0 \\ 0 & 3 & \infty & \infty & 2 \\ \infty & 3 & 12 & \infty & 0 \\ 11 & 0 & 0 & \infty & \infty \end{bmatrix}$$

As each row and column have '0', the reduced cost is 0

→ Optimum cost + reduced cost + old value M[1,4]

→ 25+0+0

→ 25

**Path [1,5] :** make 1<sup>st</sup> row and 5<sup>th</sup> column as  $\infty$  and set M[5,1]=  $\infty$  (no back tracking allowed )

$$\begin{bmatrix} \infty & \infty & \infty & \infty & \infty \\ 12 & \infty & 11 & 2 & \infty \\ 0 & 3 & \infty & 0 & \infty \\ 15 & 3 & 12 & \infty & \infty \\ \infty & 0 & 0 & 12 & \infty \end{bmatrix} \xrightarrow{i} \begin{bmatrix} \infty & \infty & \infty & \infty & \infty \\ 12 - 2 & \infty & 11 - 2 & 2 - 2 & \infty \\ 0 & 3 & \infty & 0 & \infty \\ 15 - 3 & 3 - 3 & 12 - 3 & \infty & \infty \\ \infty & 0 & 0 & 12 & \infty \end{bmatrix} \xrightarrow{i} \begin{bmatrix} \infty & \infty & \infty & \infty & \infty \\ 10 & \infty & 9 & 0 & \infty \\ 0 & 3 & \infty & 0 & \infty \\ 12 & 0 & 9 & \infty & \infty \\ \infty & 0 & 0 & 12 & \infty \end{bmatrix}$$

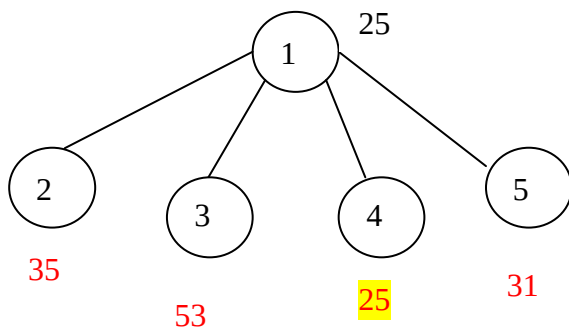
As each row and column doesn't have '0', here the minimum cost is 2 (2<sup>nd</sup> row) and 3 (4<sup>th</sup> row) are reduced

→ Optimum cost + reduced cost + old value M[1,4]

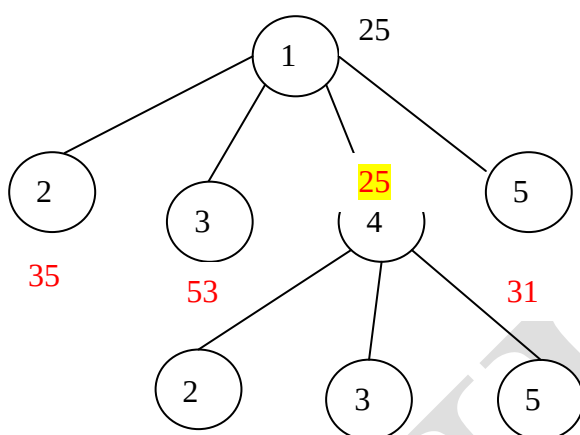
→ 25+(2+3)+1

→ 31

The state space tree will be now



The cost of node 4 is optimum, whose value is 25. now set this node as E-node and generate its children



Now the path  $1 \rightarrow 4$  and further the paths may be  $1 \rightarrow 4 \rightarrow 2$  or  $1 \rightarrow 4 \rightarrow 3$  or  $1 \rightarrow 4 \rightarrow 5$

Consider the paths  $1 \rightarrow 4 \rightarrow 2$ , now set 1<sup>st</sup> and 4<sup>th</sup> rows as  $\infty$  and 2<sup>nd</sup> and 4<sup>th</sup> column as  $\infty$ ,  
And  $M[4,1] = M[2,1] = \infty$

|          |          |          |          |          |
|----------|----------|----------|----------|----------|
| $\infty$ | $\infty$ | $\infty$ | $\infty$ | $\infty$ |
| $\infty$ | $\infty$ | 11       | $\infty$ | 0        |
| 0        | $\infty$ | $\infty$ | $\infty$ | 2        |
| $\infty$ | $\infty$ | $\infty$ | $\infty$ | $\infty$ |
| 11       | $\infty$ | 0        | $\infty$ | $\infty$ |

As each and every column and row has '0', which is minimum

Cost of node 2  $\rightarrow$  optimum cost + reduced cost + old value  $M[4,2]$

$\rightarrow 25 + 0 + 3$

$\rightarrow 28$

Consider the paths  $1 \rightarrow 4 \rightarrow 3$ , now set 1<sup>st</sup> and 4<sup>th</sup> rows as  $\infty$  and 3<sup>rd</sup> and 4<sup>th</sup> column as  $\infty$ ,  
And  $M[4,1] = M[3,1] = \infty$

|          |          |          |          |          |
|----------|----------|----------|----------|----------|
| $\infty$ | $\infty$ | $\infty$ | $\infty$ | $\infty$ |
| 12       | $\infty$ | $\infty$ | $\infty$ | 0        |
| $\infty$ | 3        | $\infty$ | $\infty$ | 2        |
| $\infty$ | $\infty$ | $\infty$ | $\infty$ | $\infty$ |
| 11       | $\infty$ | 0        | $\infty$ | $\infty$ |

The 3<sup>rd</sup> row has minimum value of 2 and 1<sup>st</sup> column has minimum value of 11 (11+2 are reduced cost)

Cost of node 3  $\rightarrow$  optimum cost + reduced cost + old value  $M[4,3]$



$$\rightarrow 25 + (3 + 11) + 12$$

$$\rightarrow 50$$

Consider the paths  $1 \rightarrow 4 \rightarrow 5$ , now set 1<sup>st</sup> and 4<sup>th</sup> rows as  $\infty$  and 5<sup>th</sup> and 4<sup>th</sup> column as  $\infty$ ,  
And  $M[4,1] = M[5,1] = \infty$

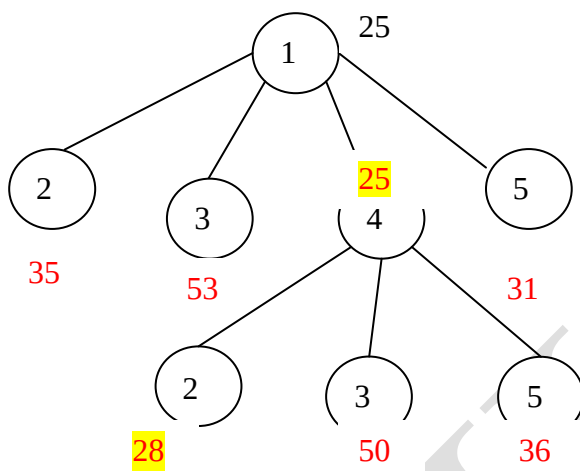
|          |          |          |          |          |
|----------|----------|----------|----------|----------|
| $\infty$ | $\infty$ | $\infty$ | $\infty$ | $\infty$ |
| 12       | $\infty$ | 11       | $\infty$ | $\infty$ |
| $\infty$ | 3        | $\infty$ | $\infty$ | $\infty$ |
| $\infty$ | $\infty$ | $\infty$ | $\infty$ | $\infty$ |
| $\infty$ | 0        | 0        | $\infty$ | $\infty$ |

The 2<sup>nd</sup> row has the minimum value of 11 (which is reduced cost)

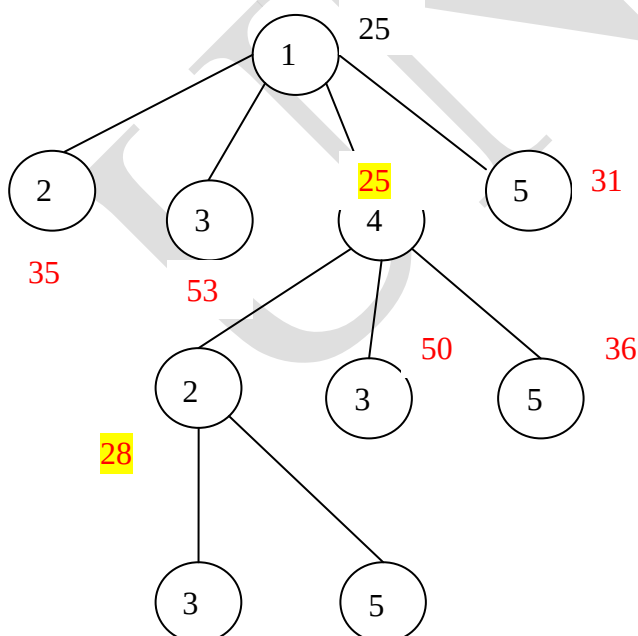
Cost of node 5  $\rightarrow$  optimum cost + reduced cost + old value  $M[4,5]$

$$\rightarrow 25 + 11 + 0$$

$$\rightarrow 36$$



The cost of node 2 is optimum, whose value is 28. now set this node as E-node and generate its children



Consider the path  $1 \rightarrow 4 \rightarrow 2 \rightarrow 3$  now.

Set 1<sup>st</sup>, 2<sup>nd</sup> and 4<sup>th</sup> rows as  $\infty$  and 2<sup>nd</sup>, 3<sup>rd</sup> and 4<sup>th</sup> column as  $\infty$

$M[4,1]=M[3,1]=M[2,1]= \infty$

|          |          |          |          |          |
|----------|----------|----------|----------|----------|
| $\infty$ | $\infty$ | $\infty$ | $\infty$ | $\infty$ |
| $\infty$ | $\infty$ | $\infty$ | $\infty$ | $\infty$ |
| $\infty$ | $\infty$ | $\infty$ | $\infty$ | 2        |
| $\infty$ | $\infty$ | $\infty$ | $\infty$ | $\infty$ |
| 11       | $\infty$ | $\infty$ | $\infty$ | $\infty$ |

The 3<sup>rd</sup> row has minimum value of 2 and 1<sup>st</sup> column has minimum value of 11 (11+2 are reduced cost)

Cost of node 3  $\rightarrow$  optimum cost + reduced cost + old value  $M[2,3]$

$\rightarrow 28 + (3 + 11) + 11$

$\rightarrow 52$

Consider the path  $1 \rightarrow 4 \rightarrow 2 \rightarrow 5$  now .

Set 1<sup>st</sup>, 2<sup>nd</sup> and 4<sup>th</sup> rows as  $\infty$  and 2<sup>nd</sup>, 4<sup>th</sup> and 5<sup>th</sup> column as  $\infty$

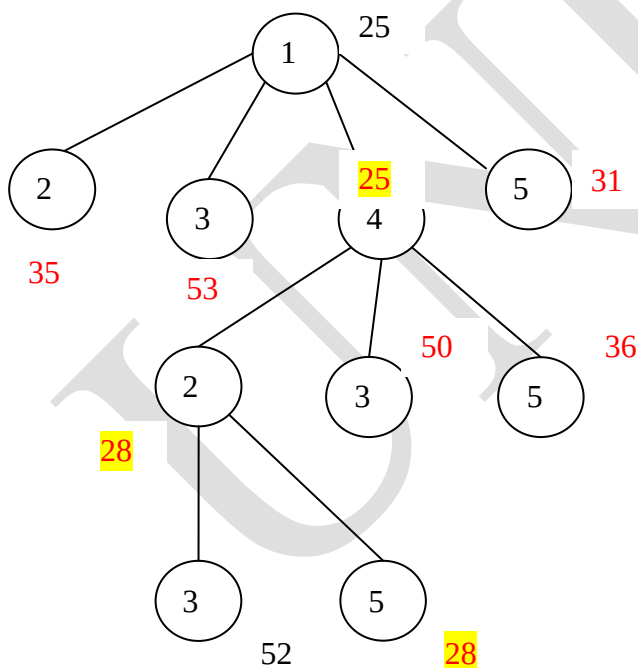
$M[5,1]=M[4,1]=M[2,1]= \infty$

|          |          |          |          |          |
|----------|----------|----------|----------|----------|
| $\infty$ | $\infty$ | $\infty$ | $\infty$ | $\infty$ |
| $\infty$ | $\infty$ | $\infty$ | $\infty$ | $\infty$ |
| 0        | $\infty$ | $\infty$ | $\infty$ | $\infty$ |
| $\infty$ | $\infty$ | $\infty$ | $\infty$ | $\infty$ |
| $\infty$ | $\infty$ | 0        | $\infty$ | $\infty$ |

Cost of node 5  $\rightarrow$  optimum cost + reduced cost + old value  $M[2,3]$

$\rightarrow 28 + 0 + 0$

$\rightarrow 28$



Now the node 5 becomes E-node and its children will be 3.

The path is now  $1 \rightarrow 4 \rightarrow 2 \rightarrow 5 \rightarrow 3$

At the node 3, we consider 1<sup>st</sup>, 2<sup>nd</sup>, 4<sup>th</sup> and 5<sup>th</sup> rows as  $\infty$  and 2<sup>nd</sup>, 3<sup>rd</sup>, 4<sup>th</sup> and 5<sup>th</sup> as  $\infty$

$$M[2,1] = M[3,1] = M[4,1] = M[5,1] = \infty$$

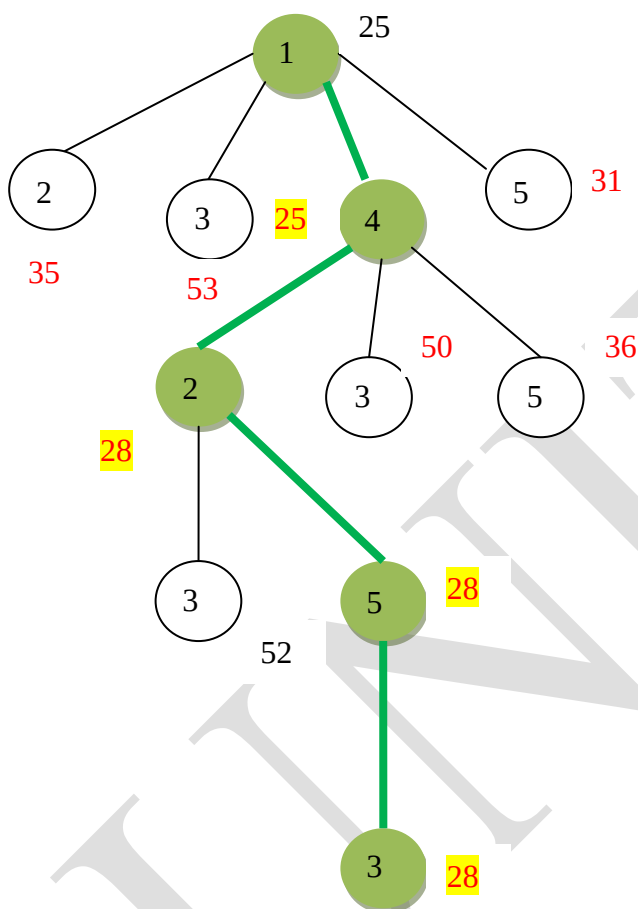
|          |          |          |          |          |
|----------|----------|----------|----------|----------|
| $\infty$ | $\infty$ | $\infty$ | $\infty$ | $\infty$ |
| $\infty$ | $\infty$ | $\infty$ | $\infty$ | $\infty$ |
| $\infty$ | $\infty$ | $\infty$ | $\infty$ | $\infty$ |
| $\infty$ | $\infty$ | $\infty$ | $\infty$ | $\infty$ |
| $\infty$ | $\infty$ | $\infty$ | $\infty$ | $\infty$ |

Cost of node 3  $\rightarrow$  optimum cost + reduced cost + old value  $M[4,5]$

$$\rightarrow 28 + 0 + 0$$

$$\rightarrow 28$$

So the tree will be now and the path will be as



### 0/1 Knapsack Problem

Consider the instance:  $M = 15$ ,  $n = 4$ ,  $(P_1, P_2, P_3, P_4) = (10, 10, 12, 18)$  and  $(w_1, w_2, w_3, w_4) = (2, 4, 6, 9)$ .

0/1 knapsack problem can be solved by using branch and bound technique. In this problem we will calculate lower bound and upper bound for each node.

Place first item in knapsack. Remaining weight of knapsack is  $15 - 2 = 13$ . Place next item  $w_2$  in knapsack and the remaining weight of knapsack is  $13 - 4 = 9$ . Place next item  $w_3$  in knapsack then

the remaining weight of knapsack is  $9 - 6 = 3$ . No fractions are allowed in calculation of upper bound so  $w_4$  cannot be placed in knapsack.

$$\text{Profit} = P_1 + P_2 + P_3 = 10 + 10 + 12$$

So, Upper bound = 32

To calculate lower bound we can place  $w_4$  in knapsack since fractions are allowed in calculation of lower bound.

$$\text{Lower bound} = 10 + 10 + 12 + (3 \times 18) = 32 + 6 = \underline{38}$$

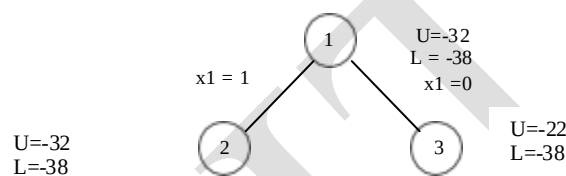
9

Knapsack problem is maximization problem but branch and bound technique is applicable for only minimization problems. In order to convert maximization problem into minimization problem we have to take negative sign for upper bound and lower bound.

Therefore, Upper bound (U) = -32

Lower bound (L) = -38

We choose the path, which has minimum difference of upper bound and lower bound. If the difference is equal then we choose the path by comparing upper bounds and we discard node with maximum upper bound.



Now we will calculate upper bound and lower bound for nodes 2, 3.

For node 2,  $x_1 = 1$ , means we should place first item in the knapsack.

$$U = 10 + 10 + 12 = 32, \text{ make it as } -32$$

$$L = 10 + 10 + 12 + 3/9 \times 18 = 32 + 6 = 38, \text{ make it as } -38$$

For node 3,  $x_1 = 0$ , means we should not place first item in the knapsack.

$$U = 10 + 12 = 22, \text{ make it as } -22$$

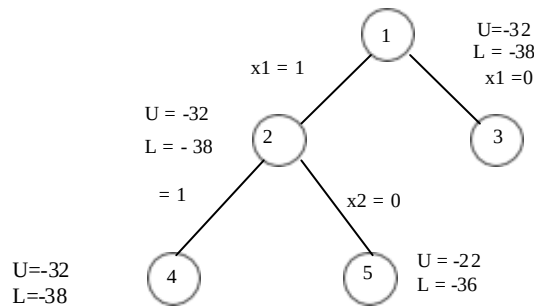
$$L = 10 + 12 + 5/9 \times 18 = 10 + 12 + 10 = 32, \text{ make it as } -32$$

Next, we will calculate difference of upper bound and lower bound for nodes 2, 3

For node 2,  $U - L = -32 + 38 = 6$

For node 3,  $U - L = -22 + 32 = 10$

Choose node 2, since it has minimum difference value of 6.

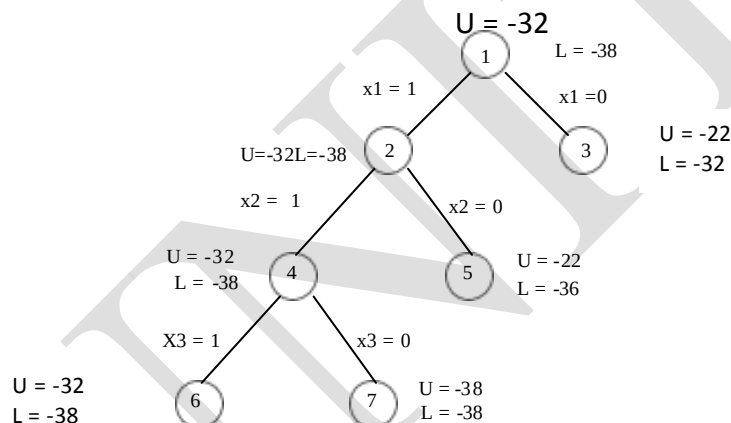


Now we will calculate lower bound and upper bound of node 4 and 5. Calculate difference of lower and upper bound of nodes 4 and 5.

For node 4,  $U - L = -32 + 38 = 6$

For node 5,  $U - L = -22 + 36 = 14$

Choose node 4, since it has minimum difference value of 6.

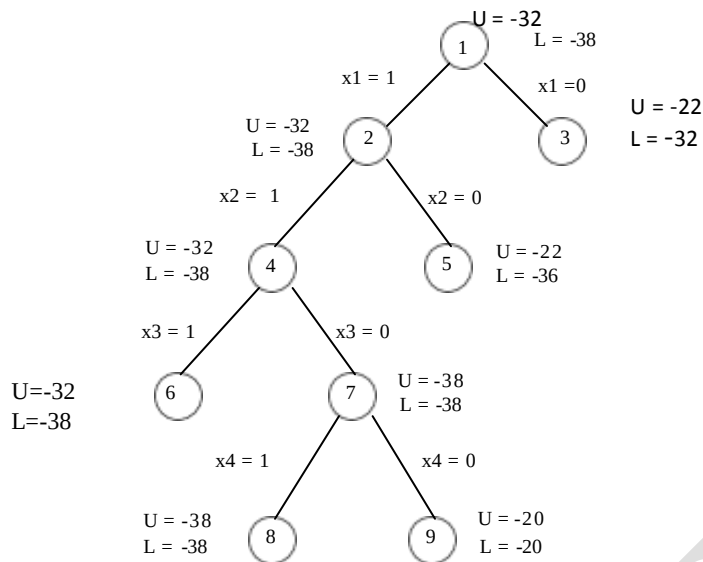


Now we will calculate lower bound and upper bound of node 8 and 9. Calculate difference of lower and upper bound of nodes 8 and 9.

For node 6,  $U - L = -32 + 38 = 6$

For node 7,  $U - L = -38 + 38 = 0$

Choose node 7, since it is minimum difference value of 0.



Now we will calculate lower bound and upper bound of node 4 and 5. Calculate difference of lower and upper bound of nodes 4 and 5.

For node 8,  $U - L = -38 + 38 = 0$

For node 9,  $U - L = -20 + 20 = 0$

Here the difference is same, so compare upper bounds of nodes 8 and 9. Discard the node, which has maximum upper bound. Choose node 8, discard node 9 since, it has maximum upper bound.

Consider the path from  $1 \rightarrow 2 \rightarrow 4 \rightarrow 7 \rightarrow 8$

$x_1=1, x_2=1, x_3=0, x_4=0$ .

The solution for 0/1 Knapsack problem is  $(x_1, x_2, x_3, x_4) = (1, 1, 0, 1)$  Maximum profit is:

$$\begin{aligned}\sum P_i x_i &= 10 \times 1 + 10 \times 1 + 12 \times 0 + 18 \times 1 \\ &= 10 + 10 + 18 = 38\end{aligned}$$